MODULE 22 · DOMAIN 3 - DEPLOYMENT, PROVISIONING, AND AUTOMATION

Deployment Strategies

Rolling, immutable, blue/green, canary - where each is implemented on AWS, and how you get back.

DOMAIN 3 of 5 EXAM WEIGHT 22% MODULE 22 of 35

admin@sthithapragna.com

Every strategy trades **capacity**, **speed** and **blast radius**. The one that matters most operationally is a fourth thing: **how fast you can get back**.

The strategies

STRATEGY	WHAT HAPPENS
All at once	Every instance updated together. Downtime, and instant rollback is impossible
Rolling	Batches updated in place. Reduced capacity while it runs
Rolling with additional batch	An extra batch launched first, so full capacity is maintained
Immutable	A full set of new instances alongside the old, then switch and terminate
Blue/green	Two complete environments; traffic switched between them
Canary and linear	A percentage of traffic to the new version, increased over time

ON THE EXAM

Immutable and blue/green both build new alongside old. The difference is whether the old environment survives after the switch.

Rolling, and its risk

- Instances are updated **in batches**, in place. Batch size controls how much capacity is unavailable at once.
- **Capacity drops during the deployment** unless you add an extra batch first. On a busy service that alone can cause errors.
- **Rollback means another rolling deployment**, which takes as long again. There is no switch to flip.
- Mixed versions serve traffic simultaneously, so both versions must tolerate each other.

> ON THE EXAM
>
> A partially completed rolling deployment is the worst state to be in: some instances new, some old, and no fast way back.

Immutable

- Launch a **complete new set of instances** alongside the existing ones, then move traffic and terminate the old.
- If the new instances **fail health checks, they are terminated and the originals are untouched**. Nothing was modified in place.
- Prevents the partially-completed state that makes a failed rolling deployment so awkward.
- Needs **double capacity** during the deployment, and takes longer because instances must launch and warm up.

> ON THE EXAM
>
> The name is the point: nothing existing is changed. A failed immutable deployment leaves the original environment exactly as it was.

Immutable or blue/green?

IMMUTABLE

Replace, then discard

- New instances join the same environment
- Old instances terminated once traffic moves
- Rollback after completion means deploying again
- One environment, temporarily doubled

BLUE/GREEN

Two environments, one switch

- Two complete environments exist independently
- Traffic switched by DNS, listener or target group
- The old environment survives, so rollback is a switch back
- Costs more: you keep both for a period

THE TIEBREAKER

Ask what happens to the old environment. Immutable terminates it; blue/green keeps it so you can switch back in seconds.

Canary and linear

- **Canary**: a small percentage of traffic to the new version, then a pause, then everything. Two steps.
- **Linear**: equal increments at a fixed interval. Ten per cent every minute, for example.
- Both need a **traffic-splitting mechanism**: weighted target groups, a weighted Lambda alias, or Route 53 weights.
- Both are only as useful as the **alarms watching them**. Shifting five per cent and not measuring it achieves nothing.

> ON THE EXAM
>
> The bake time is the feature, not the traffic split. The point is to give a problem time to appear before the rest of your users meet it.

Where AWS implements each

PLATFORM	MECHANISM
EC2 Auto Scaling	Instance refresh, with checkpoints and alarm rollback
CloudFormation	UpdatePolicy: AutoScalingInstanceRefresh, or the older AutoScalingRollingUpdate
Application Load Balancer	Weighted target groups: the raw traffic-shifting primitive
Amazon ECS	Rolling with min/max healthy percent, or native blue/green, canary and linear
AWS Lambda	Versions and aliases, with weighted routing between two versions
Elastic Beanstalk	Five deployment policies, plus environment swap for blue/green
Route 53	Weighted records, for cross-Region or cross-stack switching

ON THE EXAM

Weighted target groups on an ALB are the primitive underneath most of this. Recognise them and canary questions become simple.

EC2: instance refresh as the deployment mechanism

- **MinHealthyPercentage 100** gives launch-before-terminate, so capacity never dips. That is effectively an immutable deployment.
- **Checkpoints** with a bake time pause at chosen percentages, which is a canary in everything but name.
- **Alarm-based automatic rollback** reverses the refresh if a named CloudWatch alarm fires during it.
- Since July 2026 this is available as the **AutoScalingInstanceRefresh** CloudFormation update policy, inside the stack.

> ON THE EXAM
>
> One feature covers rolling, immutable and canary depending on how you set the percentages. The strategy is a configuration choice.

Elastic Beanstalk's five policies

- **All at once** causes downtime. **Rolling** reduces capacity. **Rolling with additional batch** maintains full capacity.
- **Immutable** launches a full set of new instances in a **separate Auto Scaling group**. Failures leave the originals untouched.
- **Traffic splitting** is immutable plus a percentage of traffic for an evaluation period, with automatic rollback. It **requires an ALB**.
- **Blue/green on Beanstalk is a separate concept**: clone the environment and **swap the CNAMEs**. It is not a deployment policy.

> ON THE EXAM
>
> Traffic splitting is Beanstalk's canary. Blue/green is the CNAME swap, and the two appear in different parts of the console for that reason.

New: ECS native blue/green

- **Launched July 2025.** Blue/green is built into ECS. Set the deployment controller to **ECS**, not CODE_DEPLOY.
- **October 2025** added **canary and linear** strategies natively, reaching parity with CodeDeploy.
- Includes **bake time**, **Lambda lifecycle hooks** for validation, and **CloudWatch alarm rollback**, all inside ECS.
- AWS's own guidance now names **ECS-native as the default choice for new deployments**, rather than CodeDeploy.

> ON THE EXAM
>
> Every course written before mid-2025 says ECS blue/green requires CodeDeploy. It does not, and AWS no longer recommends that path for new work.

ECS deployment safety

- **Minimum and maximum healthy percent** bound a rolling update, exactly as MinHealthyPercentage bounds an instance refresh.
- The **deployment circuit breaker** fails a deployment when tasks cannot reach a steady state, and can roll back automatically.
- **A test listener** lets you send synthetic traffic to the green environment before real users reach it. A dark canary.
- Native blue/green needs an IAM role permitting ECS to modify the load balancer, since it moves listener weights itself.

> ON THE EXAM
>
> The circuit breaker catches tasks that will not start. Alarms catch tasks that start and then misbehave. You want both.

Lambda deployments

- A **version** is immutable once published. An **alias** is a movable pointer to a version.
- An alias can be **weighted between two versions**, sending a percentage to each. That is the traffic-splitting primitive.
- Rollback is **repointing the alias**, which is instant. There is nothing to redeploy.
- Event sources and API Gateway should target the **alias**, never a version number, or the pointer stops meaning anything.

> ON THE EXAM
>
> Lambda has the fastest rollback of anything in this module: change where a pointer points. Compare that with a rolling redeploy of 200 instances.

What actually makes rollback fast

- **The old version still exists.** Blue/green and weighted aliases keep it; rolling and immutable have discarded it.
- **The switch is one operation.** A listener weight, an alias, a DNS record. Not a redeployment.
- **Something is watching.** An alarm that triggers rollback beats a human noticing, especially at 3am.
- **The database has not moved on.** This is the constraint nothing else can undo, and it is covered next.

> ON THE EXAM
>
> Rank strategies by rollback speed and blue/green and weighted aliases win. Rank by cost and they lose. That is the whole trade.

The database is not rollable

- You can switch traffic back in seconds. You cannot un-run a migration that dropped a column.
- **Expand and contract**: add the new structure, deploy code writing to both, backfill, deploy code reading the new, only later remove the old.
- Every intermediate state must work with **both versions of the application**, which is what makes rollback possible at all.
- Deploying a schema change and an application change together removes your ability to roll back either.

> ON THE EXAM
>
> Blue/green gives you a fast switch back. It gives you nothing if the green environment has already changed the shared database.

Separating deploy from release

- **Feature flags** let you deploy code with a feature switched off, then turn it on separately.
- **AWS AppConfig** manages flags and configuration with its own **deployment strategies**: growth rate and bake time.
- AppConfig can watch **CloudWatch alarms** during a configuration rollout and **roll the configuration back** automatically.
- Turning a flag off is faster than any deployment, and does not require the previous artefact to exist.

> ON THE EXAM
>
> This decouples the risky moment from the deployment. The code ships on Tuesday; the feature is enabled on Thursday, to ten per cent of users.

> A payments API on ECS Fargate must be updated with zero downtime. If error rates rise after the new version takes traffic, the team must be able to return to the previous version within one minute. They also want to run automated validation against the new version before any customer reaches it.
>
> The team is building this in 2026 and has no existing CodeDeploy configuration.

What should the team implement?

A. ECS native blue/green with a test listener, bake time and CloudWatch alarm rollback

B. ECS blue/green using CodeDeploy with an appspec file and a deployment group

C. An ECS rolling update with minimum healthy percent set to 100

D. Deploy to a second ECS cluster and switch Route 53 weighted records

◆ PAUSE AND DECIDE

What should the team implement?

A ECS native blue/green

The test listener gives pre-customer validation, bake time gives the observation window, and alarm rollback returns traffic in seconds.

B Blue/green via CodeDeploy

Works, and since July 2025 it is extra machinery: a CodeDeploy application, deployment group, appspec and role. AWS now recommends native for new work.

C Rolling at 100 percent

Avoids the capacity dip and discards the old tasks as it goes, so rollback means another full deployment. Not one minute.

D Second cluster plus Route 53

Achieves blue/green at the DNS layer, where TTLs and resolver caching make a one-minute switch back optimistic. Module 14 covered why.

THE SIGNAL

"No existing CodeDeploy configuration" and "building this in 2026" together point at the native path. Option B is not wrong, it is dated.

> A team performs a blue/green deployment. The green environment runs a database migration that renames a column and drops the old one. Twenty minutes after the switch, a serious bug is found in the new application version.
>
> The team switches traffic back to blue, and blue immediately begins failing.

What went wrong, and what should have been done?

A. The blue environment's instances had been terminated automatically

B. The schema change was not backwards compatible. An expand-and-contract migration would have kept blue working

C. The load balancer needed connection draining configured

D. The switch should have used Route 53 rather than target group weights

◆ PAUSE AND DECIDE

What went wrong, and what should have been done?

A **Blue was terminated**

The scenario states traffic was switched back to blue, so blue clearly still existed. It ran and then failed.

B **Non-backwards-compatible schema change**

Blue's code queries a column that no longer exists. Expand and contract keeps both readable until the old version is retired.

C **Connection draining**

Governs in-flight requests during a switch. It has nothing to do with blue failing consistently afterwards.

D **Use Route 53 instead**

Changes the switching mechanism, and the switch worked. What failed was the application once traffic arrived.

THE SIGNAL

The deployment strategy was correct and the migration was not. Blue/green protects the application layer; nothing protects a shared database from a destructive change.

> A Lambda function behind API Gateway serves 40,000 requests per minute. The team wants to expose a new version to 5 percent of traffic for ten minutes. They will monitor an error-rate alarm, and shift the rest only if it stays clear. Rollback must be immediate.
>
> The function is invoked through an alias.

What should the team use?

A. Weighted routing on the Lambda alias between the two versions, with an alarm watched during the bake period

B. Deploy the new code over the existing version and monitor afterwards

C. Two API Gateway stages with a Route 53 weighted record between them

D. An ALB with weighted target groups in front of two Lambda functions

◆ PAUSE AND DECIDE

What should the team use?

A Weighted alias routing

Built for exactly this: two immutable versions, a percentage split, and rollback by repointing the alias, which is instant.

B Deploy over the existing version

Published versions are immutable, so this creates a new one. It also gives no traffic split and no fast rollback.

C Stages plus Route 53 weights

Works and shifts at the DNS layer, where caching makes both the split and the rollback imprecise and slow.

D ALB with weighted target groups

Valid for Lambda targets, and it introduces a load balancer where API Gateway already fronts the function. Extra components for no gain.

THE SIGNAL

The alias is already there and already the invocation target. Weighting it is a configuration change, not an architecture change.

> An Auto Scaling group of 40 instances runs a service at 70 percent utilisation. A rolling deployment with a batch size of 25 percent causes elevated latency and occasional 503 errors every time it runs.
>
> The team must keep full capacity throughout, pause after the first quarter for review, and reverse automatically if error rates rise.

What should the team configure?

A. An instance refresh with MinHealthyPercentage 100, checkpoints at 25 percent, and an alarm specification

B. Reduce the rolling batch size to 10 percent

C. The AutoScalingRollingUpdate CloudFormation policy with MaxBatchSize 4

D. Increase the group's desired capacity before deploying, then reduce it afterwards

◆ PAUSE AND DECIDE

What should the team configure?

A Instance refresh, 100 percent, checkpoints, alarms

Launch-before-terminate keeps capacity, checkpoints give the review pause, and the alarm specification gives automatic reversal. All three requirements.

B Smaller batches

Reduces the capacity dip without removing it, and makes the deployment far longer. It also offers no pause and no rollback.

C AutoScalingRollingUpdate

Terminates before launching, so capacity still drops, and it has neither checkpoints nor alarm-based rollback.

D Over-provision manually

Solves capacity by hand for one deployment, with two manual steps to forget, and answers neither of the other requirements.

THE SIGNAL

Three requirements, and one feature covers all three by configuration. This is the module's point: the strategy is a setting, not a separate tool.

> A team wants to release a redesigned checkout flow to 10 percent of users. They will monitor conversion and error rates for a week, then expand or withdraw it based on results. The code for both flows is already deployed and passing tests.
>
> Withdrawing the feature must not require a deployment.

What should the team use?

A. AWS AppConfig feature flags, with a deployment strategy and CloudWatch alarm rollback

B. A canary deployment shifting 10 percent of traffic to a new version

C. Blue/green with the green environment serving 10 percent

D. Two Auto Scaling groups behind weighted target groups

◆ PAUSE AND DECIDE

What should the team use?

A AppConfig feature flags

The code is already deployed. A flag turns the feature on for a subset and off again instantly, with no deployment either way.

B Canary deployment

The closest competitor, and it splits traffic between versions. Both flows are already in the same deployed version, so there is nothing to split.

C Blue/green at 10 percent

Means running two full environments for a week, and withdrawing still means a traffic switch rather than a configuration change.

D Two ASGs, weighted

Doubles the fleet for a week to serve code that is already present in the existing fleet.

THE SIGNAL

"The code for both flows is already deployed" is the clause. Nothing needs deploying, so no deployment strategy is the answer.

The five things to carry forward

- **Immutable terminates the old; blue/green keeps it.** That is what makes blue/green rollback a switch.
- **Instance refresh is rolling, immutable or canary depending on configuration.** The strategy is a setting.
- **ECS blue/green is native since July 2025**, with canary and linear since October. CodeDeploy is no longer the default.
- **A Lambda alias is the fastest rollback here**: move a pointer, no redeployment.
- **No deployment strategy can undo a schema change.** Expand and contract, or lose your ability to roll back.

> ON THE EXAM
>
> Next: event-driven operations - Lambda, S3 notifications, Step Functions, service quotas and tagging.